NiryoStudio Guide

Oct 23, 2026 · @Emranul Hoque

What every page and button in NiryoStudio 1.16.3 does, in plain words, and which ones Team M needs for the Ned2 puck-pushing project.

The screen

NiryoStudio has three areas: a menu on the left, the page you picked in the middle, and a 3D copy of the robot on the right. The 3D robot always shows what the real robot *thinks* its pose is; if the two disagree, the robot's calibration is off.

Area	What it holds
Left menu (click the N logo or the icons to expand it)	Dashboard, Programs, Direct move, Equipment, Library, My robot, Updates. Greyed-out items need a connected robot.
Top bar	A **Physical / Simulation** switch (real robot or a virtual one), the connection status (red dot = not connected), the robot's name once connected, and **Connect Robot**.
3D view	The digital twin. Tick boxes show the **Base** frame (the robot's fixed origin) and the **TCP** frame (the tool tip). **New Frames** adds your own reference frames.
Position informations panel (bottom of the 3D view)	Live numbers: the six joint angles, the tool-tip position (X, Y, Z in mm) and orientation (roll, pitch, yaw), and the TCP speed. Copy icons put them on the clipboard.
Bottom-left menu (the three dots)	Language, **Radians / Degrees** for all angle fields, light/dark theme, fullscreen, account, feedback.

Units: distances are millimetres, angles are radians unless you switch to degrees in the bottom-left menu. Our code uses radians, so leave it on radians to avoid mix-ups.

Connecting and calibrating

Every session starts the same way: power on, wait for the ring, connect, calibrate. Calibration is needed after **every** power-on or reboot, and the robot refuses to move until it is done.

Connect robot (top bar or Dashboard) opens a list of robots found on the network. Ours is `e8-dfb-e17`. If it isn't listed, type the address by hand: `169.254.200.200` on our cable, `10.10.10.10` on the robot's own Wi-Fi.

Connection modes (My robot → Settings → Robot connection → Edit):

Mode	Meaning	For us
Auto (factory default)	Starts as Direct Ethernet; switches to DHCP if a router answers	Fine as it is
Direct Ethernet	A cable straight to the laptop, fixed address 169.254.200.200	**What we use**
Ethernet (DHCP) / Manual IP	The robot joins a wired network	Not needed
Hotspot	The robot's own Wi-Fi "Niryo Hotspot …", password `niryorobot`	Fallback only; the laptop loses internet
Wi-Fi	The robot joins an existing Wi-Fi	Plan B with a phone hotspot

Calibration. A flashing **Calibration needed** button appears on the Dashboard, on My robot and on Direct move. Click it once; the robot drives each joint to its end stop by itself, about a minute, then shows "Calibration OK". Clear space is a must. If a joint doesn't move, the button comes back as "Calibration needed": that's what happened to our elbow.

What the LED ring means (current Ned2 software; older manuals describe the same states in blue)

Ring	Meaning
White	Starting up, wait
Green, breathing	Calibrated and ready, motors holding
Green, steady	A program is running
Green, moving	A program is paused
Blue, moving (chase)	Calibration needed

Ring	Meaning
Blue, snake	Calibration running
Blue, breathing	FreeMotion: motors released
Orange, flashing	Collision detected: check the obstacle, give a new command
Orange, steady	A joint is outside its limits: calibrate
Red, flashing	Motor error: check Hardware status
Red, steady	Fatal error: read the Robot logs, restart

Dashboard

The home page. Nothing on it moves the robot except the calibration card and the Play icons on program cards.

Card	What it does
Connection status	The robot's name, whether it is connected, in which mode, and a **Disconnect** button.
Calibration	The flashing **Calibration needed** button (see above). Shows "Calibration OK" when done, then becomes **New calibration**.
Update announcement	Appears when a newer NiryoStudio exists; clicking installs it. Ours is current.
My connected equipments	The accessories the robot knows about, with a **New Scan** button. The robot does not notice a newly plugged gripper by itself: scan.
My latest programs	Recent Blockly or Python programs. The **Play** icon on a card runs it, so the robot moves.
Adverts	"Interactive 3D Simulation" and "PID control" are paid extras. We don't need them.

Direct move

This is the page for moving the robot by hand from the computer, and the one we'll use most. It has two tabs. **Everything on this page can move the robot**, so clear the table first.

Move tab (set a target, then go there)

Control	What it does
Joints sliders, J1 Base … J6 Hand Rotation	Set a target angle for each joint. The **−** and **+** buttons nudge by a small step; you can also type a value. Nothing moves yet.
Pose: Translation X, Y, Z and Rotation Roll, Pitch, Yaw	The same target expressed as where the tool tip should be (mm) and how it should be tilted (rad). The robot works out the joint angles itself.
Reachable solution (green tick)	The target is inside the arm's reach. A red mark means the robot cannot get there, so Play will refuse.
Current position	Resets all sliders to where the arm actually is. Click this before Play if you aren't sure what is queued.
Select position	Load a position saved in the Library and move the robot to it. The robot moves.
Save position	Store the current slider values in the Library under a name.
Arm speed	A cap on how fast every move on this page runs, in % of maximum. Keep it low (20 %) while learning; 100 % is fast.
Home position	Moves the arm to its folded resting pose. **The robot moves.**
Play	Moves the arm to the slider target. **The robot moves.**

Jog tab (move a bit at a time, like a joystick)

Hold a **−** or **+** button and that one joint turns, or the tool tip slides along one axis, until you let go. There is no Play: **the robot moves as soon as you hold a button.** Each section has its own speed slider (Joints, Translation, Rotation); the Arm speed slider at the bottom is greyed out on this tab. Useful for lining up the pusher above the line without typing numbers.

FreeMotion (learning mode). The motors release so you can move the arm by hand, and the 3D view follows. In NiryoStudio 1.16 there is no on-screen switch for it: you **hold the FreeMotion button on the robot's wrist panel**, or our code turns it on with

`set_learning_mode(True)`. This is how we'll record demonstrations. Support the arm when the motors let go, and keep fingers out of the joints.

Programs and Library

Programs is NiryoStudio's own way of programming the robot, in two flavours. We write our code outside the app, so we'll rarely use it; it's still handy for a quick test.

Tab	What it is
Blockly	Drag-and-drop blocks (move here, open gripper, wait…). **Play** converts them to Python, sends them to the robot and runs them; Play turns into **Stop** while running. **Home position** moves the arm. **Save** / Autosave keep the program on the laptop; Import and Export use JSON files.
Python	A code editor for `pyniryo` scripts, with Play, Stop and an error pop-up showing the stack trace. Autocomplete works only when connected. The robot does not return home by itself when a program ends.
Add Position or Trajectory	A side panel that turns the robot's *current* pose into a block. The robot's physical **Save** button does the same: a short press saves a pose, a long press records a trajectory while you move the arm by hand (stored on the robot).

Library is the store of everything saved, in the app or on the robot. Every **Play** here moves the robot.

Section	Holds
Programs	All saved programs: open, run, or convert Blockly to Python.
Positions	Named poses, from programs or from Direct move's Save position. Play moves the robot there.
Trajectories	Recorded motions. Play replays one; two extra buttons go to the trajectory's first point or to home. Niryo's recorder samples only 7 times a second, so our own recorder replaces it.

Section	Holds
User frames	Your own coordinate frames, e.g. one aligned with the path sheet. **Create new** guides you through it.
Workspaces	Vision-set work areas defined by 4 printed markers. Clicking a marker on a card moves the robot to it, so no gripper attached when testing.
Sounds	Sounds stored on the robot that programs can play.

Equipment

Everything plugged into the robot. The robot only knows about an accessory after a **scan**.

Tab	What it does
Accessories	One card per detected device. **New Scan** makes the robot look for newly plugged equipment; run it after attaching the gripper (software 6.1 also scans by itself after a tool change). Grippers plug into the magnetic wrist mount plus the small plug on its right; you may plug and unplug while the robot is on. Niryo tools only; custom 3D-printed jaws for the Custom Gripper are allowed.
Vision (appears only with Niryo's Vision Set)	The live camera picture with brightness, contrast, saturation and colour filters, and a wizard that defines a **workspace** from 4 printed markers. On Wi-Fi the stream can lag or freeze: use the cable. Clicking a marker moves the robot.
Conveyor belt	Speed, direction, run/stop for up to two Niryo conveyors. We have none.
Input / Output	The robot's electrical pins, each HIGH or LOW. DI4 and DO4 are reserved for the electromagnet, DI5 for the conveyor's infrared sensor. Not needed for us.

For our project the Accessories scan matters most: our check script reported "no tool" while a gripper was fitted, which is exactly what an unscanned tool looks like.

My robot

The maintenance page. Nothing here moves the arm except the calibration.

Top bar: the robot's name (pencil icon renames it), a **Calibration** button that shows "Calibration needed" when the robot has lost its joint zeros and "Calibration…" while one runs, and **Shutdown** / **Reboot** buttons (shut down from here, then press the power button; never just unplug it).

Information tab

Item	What it tells you
Raspberry Pi	Free space on the robot's SD card. **Purge ROS logs** deletes old log files to free space; the toggle purges them at every start.
Application version	Your NiryoStudio version (1.16.3).
Robot software version	The robot's own software (ours: 6.1.2-b374). PyNiryo 1.2 needs 5.5.0 or newer.
App logs / **Robot logs**	Open the log files. Robot logs is a window with Info / Warn / Error filters and an "Enable Log" box that saves the robot's messages to the laptop (off by default; ours is now on). It is what we read after a problem such as a failed calibration.
Hardware status	A drawing of the arm with, per joint, the firmware version, temperature and voltage. Green text is normal; red means a fault.
Tool reboot	Power-cycles the tool on the wrist (gripper) without touching the arm.
Reboot motors	Restarts the six motors' controllers. The first thing to try when a joint does not respond, before a new calibration.

Settings tab

Item	What it does
Robot connection	Shows the current mode and IP (copy icon). **Edit** opens the network dialog with Wi-Fi / Ethernet / Hotspot tabs; Apply usually drops the connection, so reconnect afterwards.

Item	What it does
Arm speed	The robot's overall speed cap in %. Keep it low while testing.
Sound volume	The robot's speaker.
Home position	Define your own home pose: Current position, Select position, Play (moves there), trash resets to the factory home.
Program autorun	Which program the robot's top button runs (factory: a demo), once or looped, or disabled. **Disable it** so a stray press never starts the demo near our set-up.
Modify axis PID	A paid tuning screen for joint 5. Niryo warns it can damage the motor. Not for us.

Updates, Simulation, PID and preferences

Updates (left menu): installed versions and available updates for NiryoStudio, the 3D simulation stack and the robot software. For robots on 6.0 or newer the package downloads to the laptop once, then **Find robots** installs it on the robot over the network; never power off during that. Ours are current (app 1.16.3, robot 6.1.2), so leave it alone.

Simulation (the Physical / Simulation switch, Launch Simulation, Try for Free): a virtual Ned2 on the laptop. The free version allows 10 instructions per program and no `pyniryo`; the full one is a paid subscription and needs 16 GB RAM and 25 GB disk. Connecting to the real robot stops it. Not needed for us.

PID control (My robot → Settings): tuning of joint 5's motor controller, paid, with a warning that it can damage the motor. Not for us.

Preferences (bottom-left menu): language (English or French), **Radians / Degrees**, light or dark theme, fullscreen, account, feedback. **SSH terminal** (the black bar at the bottom) opens a command line on the robot's computer; advanced only.

Account: NiryoStudio asks for a Niryo account or a guest token at first start, and works offline after that.

What we use, and when

Our project runs from our own Python code (`pyniryo`), so NiryoStudio is the cockpit, not the engine. We use it to check, to fix, and to stop.

Our step	Page in NiryoStudio	Why
Every session: power on, connect, calibrate	Top bar, My robot	Connect, run the calibration if asked, read the hardware status before any movement.
Step 4: first movements	Direct move	Set Arm speed to 20 %, Current position, Home position: see the arm move safely before our code does.
Camera and pusher set-up	Equipment	Scan for the gripper so the robot knows it is there; check the camera stream if it is Niryo's Vision Set.
Recording demonstrations	Direct move (FreeMotion)	Switch learning mode on so the arm can be guided by hand while our recorder logs it. Our own recorder replaces the app's 7 Hz trajectory saving.
Workspace and test set-up	Library, 3D view	Save the home pose and the sheet corners as named positions; add a frame for the sheet if useful.
Any time a joint misbehaves	My robot	Reboot motors, then Calibration, then Robot logs; write the result in the meeting note.
Not needed for us	Programs, Updates, Simulation, PID	We write Python ourselves; the robot software is already current; the simulation needs a paid plan.

Stopping the robot, and what not to touch

From softest to hardest. Use the softest one that does the job.

1. **Stop in NiryoStudio.** Ends the current program or movement. The arm stays where it is, motors holding. Use this first.

2. **Top button on the robot, held for more than 2 seconds.** Stops a running program from the robot itself. A short press plays or pauses the saved program, so don't tap it by accident.
3. **Shutdown in My robot.** Not a stop: a clean power-off after the arm is in the home pose. Use it at the end of a session, then press the power button.
4. **Red emergency stop on the safety box.** Cuts motor power instantly. The arm has no brakes, so it **drops** (cushioned). Only for danger, and never with a hand under the arm.

Don't touch unless we've agreed it:

- **Reboot** and **Shutdown** during a movement: it corrupts the SD card.
- Anything under Settings that changes motor limits, speed or acceleration: the course forbids changing the robot's configuration.
- **Play** on the Direct move page while the sliders hold a target you didn't set.
- The network mode, once the cable works.

Sources

- NiryoStudio documentation, read page by page on 23.10.26: overview, connection, direct move, library, Blockly, Python, equipment, vision, My robot, updates, simulation, PID
- Ned2 user manual: LED ring, control panel, safety
- What we saw on our own robot `e8-dfb-e17` with NiryoStudio 1.16.3 on 23.10.26